OPERATING PROCEDURE

Generative AI-Based Intelligent Automation Testing System

PROJECT AREA	TECHNOLOGY / DESCRIPTION
AI Testing Engine	Python / Node.js, Google Gemini 2.5 Flash, Groq LLM, IEEE 829 Schema
Testing Dashboard	Browser-based TypeScript, React 19, Tailwind CSS v4, Lucide Icons
Backend API Server	Node.js, Express.js, MongoDB Atlas (Mongoose ODM), JWT, bcrypt
Optimization & Quality	Levenshtein & Jaccard Deduplication, Multi-Factor Prioritization, RTM
Primary Use	Autonomous requirement ingestion, ambiguity audit, test execution & CI/CD

This document explains how to prepare, start, operate, demonstrate, and stop the Generative AI-Based Intelligent Automation Testing System project. It covers the GenAI requirement processing workflow, the backend API services, the browser-based testing dashboard, and the live runner, including the main processing stages and how to interpret their outputs.

Project Operating Guide

1. Purpose

The Generative AI-Based Intelligent Automation Testing System is an autonomous quality engineering platform designed to eliminate manual bottlenecks in the software testing lifecycle. It combines an AI-assisted requirement analysis and IEEE 829 test generation workflow with an execution and CI/CD environment for end-to-end verification.

2. System Flow

STAGE	OPERATION
1. SRS Ingestion	Raw requirement documents (PDF, Word, or plain text) provide structured specifications.
2. Ambiguity Detection	The linguistic NLP engine detects untestable phrases and suggests concrete metrics.
3. Test Case Synthesis	Google Gemini 2.5 Flash generates IEEE 829 positive, negative, and edge test cases.
4. Test Data Generation	The synthetic engine generates realistic, privacy-compliant mock data (Luhn credit cards, boundary numbers).
5. Optimization & RTM	The system removes duplicate tests, prioritizes P1/P2/P3 suites, and maps requirement traceability.
6. Script Gen & Execution	Generates Playwright/Selenium automation code and executes tests live in a sandboxed runner.

3. Requirements Before Starting

- Node.js (v18 or higher) and npm installed and available from the terminal.
- Git installed if the testing system repository needs to be cloned.
- A valid Google Gemini API key or Groq API key configured in the backend environment.
- MongoDB Atlas connection string configured for test case, history, and telemetry persistence.
- Backend dependencies installed in the `server/` directory containing `package.json`.
- Frontend dependencies installed in the root application directory containing `package.json`.

4. Backend API Server – Starting the System

Step 1 — Open the backend folder

Open the project in a terminal and enter the server directory:

```
cd server
```

Step 2 — Install backend dependencies

Install all required Express, Mongoose, JWT, and AI client libraries:

```
npm install
```

Step 3 — Configure the environment variables

Create a local `.env` file in the server root and add the database URI and AI keys:

```
PORT=5001
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/genai_testing
JWT_SECRET=your_jwt_secret_key
GEMINI_API_KEY=your_gemini_api_key_here
```

Keep the API key private. Do not place real credentials in reports or public repositories.

4. Backend API Server – Starting the System (Continued)

Step 4 — Start the Express backend server

Open a terminal in the root directory of the server project and run:

```
npm run dev
```

Keep this terminal open while using the API. The server runs on port 5001 and automatically reloads when source files change.

Step 5 — Open the API health endpoint

Open the following address in a browser to verify backend operational readiness:

```
http://localhost:5001/api/health
```

The health endpoint returns operational status, connected database confirmation, and active AI engine details.

5. Backend REST API – Normal Operating Procedure

STEP	OPERATION
1. Verify API Health	Confirm backend is operational at `/api/health` or live URL `https://genai-testing-system-api.onrender.com/health`.
2. Select Endpoint	Choose desired testing route: `POST /api/ai/generate-tests`, `POST /api/ai/analyze-ambiguity`, or `POST /api/ai/prioritize-tests`.
3. Enter the Request	Provide the requirement specification text and test category array in JSON format.
4. Execute Request	Submit the HTTP POST request with Bearer JWT token header and wait for AI synthesis.
5. Review Response	Inspect the generated test cases, preconditions, steps, expected results, and priority scores.

Example API Request

```
{
  "requirements": "User must log in with email and password. System shall respond within 200ms.",
  "categories": ["functional", "boundary", "security"]
}
```

6. Understanding the AI Engine Output

FIELD	DESCRIPTION
testCases	List of generated structured test case objects conforming to IEEE 829.
id	Unique test identifier assigned to the scenario (e.g., `TC-001`, `TC-002`).
scenario	Concise title and behavioral description of the functional condition under test.
priority	Assigned execution priority (`P1` Critical, `P2` High, `P3` Medium) based on multi-factor scoring.
preconditions	Prerequisite system state required before test execution (e.g., user account exists).
steps	Sequential array of actionable testing instructions to be executed by tester or script.
expectedResults	Clear, verifiable condition defining whether the test passes or fails.
ambiguities	List of detected untestable phrases with severity ratings and suggested measurable rewrites.

7. AI Testing Engine – Internal Workflow

The AI Testing Engine utilizes a multi-stage deterministic pipeline. Each stage performs a specialized quality task and passes verified JSON structures to the subsequent phase.

STAGE	FUNCTION
Document Extraction	Parses raw PDF/DOCX/TXT files into normalized text buffers and extracts requirement clauses.
Ambiguity Evaluation	Evaluates text against non-verifiable linguistic patterns and suggests testable metrics.
Structured Synthesis	Generates IEEE 829 positive, negative, and boundary test cases via Google Gemini 2.5 Flash.
Optimization & Deduplication	Calculates Levenshtein and Jaccard similarity to eliminate redundant test cases.
Traceability & Prioritization	Constructs bidirectional RTM and partitions tests into Smoke (P1), Sanity, and Regression suites.

Main Project Files

FILE / FOLDER	RESPONSIBILITY
server/src/server.js	Bootstraps the Express application, configures security headers, CORS, and mounts routes.
server/src/services/aiService.js	Coordinates Gemini/Groq LLM calls, prompt templates, and zero-downtime heuristic fallback.
server/src/routes/aiRoutes.js	Exposes 12 REST API endpoints for test generation, ambiguity audit, and script synthesis.
src/components/Dashboard.tsx	Renders executive KPI metrics, live execution telemetry, charts, and quality radar.
src/components/SRSInput.tsx	Handles document upload (PDF/DOCX), prompt engineering, and test case generation.
src/lib/store.ts	Zustand global state store managing active test suites, projects, and persistence.

8. Testing Dashboard – Starting the System

The Testing Dashboard is a modern React 19 single-page application built with TypeScript, Vite, and Tailwind CSS.

Step 1 — Open the project folder

Open the root project folder in your code editor or terminal. Locate the directory containing `package.json`.

Step 2 — Open a terminal

Open a terminal at that application directory. Make sure commands are executed from the folder containing `package.json`.

Step 3 — Install dependencies (first-time setup)

```
npm install
```

Step 4 — Start the development server

```
npm run dev
```

Keep the terminal open. The development server prints a local address. Open that address in a web browser to load the Testing Dashboard.

9. Testing Dashboard – Normal Operating Procedure

STEP	OPERATION
1. Open Application	Load the local address printed by development server or live URL `https://generative-ai-based-intelligent-aut-delta.vercel.app/`.
2. Review Workspace	Sign in using Demo Login, Google SSO, or credentials. Select project in ProjectSwitcher.
3. Ingest Requirements	Navigate to **"SRS Input & Test Case Generator"**. Upload a PDF/Word file or paste requirement text.
4. Audit Ambiguities	Open **"Ambiguity Detection"**. Review flagged vague terms and click **"Replace in Requirement"** to apply testable metrics.
5. Generate Test Suite	Click **"Generate Test Cases"**. Review synthesized IEEE 829 test cases with preconditions, steps, and expected results.
6. Synthesize Edge Data	Navigate to **"Negative & Edge Cases"** for BVA/security tests, and **"AI Test Data Generator"** for Luhn-verified mock datasets.
7. Deduplicate & Prioritize	Run **"Duplicate Detection"** to eliminate redundancies. View **"Test Prioritization"** to inspect P1/P2/P3 regression rankings.
8. Traceability & Impact	Check **"Requirement Traceability Matrix"** to verify 100% coverage. Use **"Requirement Change Impact"** when specifications change.
9. Script Gen & Execution	Open **"Automation Script Generator"** to export Playwright/Selenium POM code. Run tests in **"Live Test Execution"** to observe telemetry.

Operational User Journey

Complete Lifecycle Execution Sequence:

Step 1: Upload Document → **Step 2**: Auto-Detect Ambiguities → **Step 3**: Auto-Generate Structured Tests → **Step 4**: Generate Negative & Edge Cases → **Step 5**: Synthesize Synthetic Data → **Step 6**: Deduplicate Suite → **Step 7**: Prioritize Tests & Verify RTM → **Step 8**: Generate Playwright Code → **Step 9**: Execute in Live Runner.

All generated assets synchronize automatically with MongoDB Atlas and reflect immediately on the Testing Intelligence Dashboard.

10. Core Methodologies & Algorithmic Operation

Boundary Value Analysis (BVA) & Equivalence Partitioning (ECP)

The system automatically applies formal black-box testing methodologies to eliminate boundary flaws. It derives boundary partitions (Min - 1, Min, Max, Max + 1) and equivalence classes from requirement clauses, synthesizing targeted test cases for numerical limits, string lengths, and buffer capacities.

Mathematical Test Deduplication

To prevent suite bloat and wasted CI/CD execution cycles, the engine normalizes test steps and computes pairwise similarity using Levenshtein edit distance and Jaccard token intersection:

```
Jaccard Similarity(A, B) = |A ∩ B| / |A ∪ B|
```

Candidate test pairs with similarity scores ≥ 75% are flagged for review, while pairs ≥ 90% are classified as redundant, enabling 1-click merging or pruning.

Three-Pass Prioritization Calculation

PASS	OPERATION
Pass 1 — Multi-Factor Scoring	Calculates priority score: `(0.50 * Business Impact) + (0.30 * Defect Probability) - (0.20 * Execution Cost)`.
Pass 2 — Suite Partitioning	Partitions tests into Smoke Suite (Top 10% P1 tests for <2 min CI runs), Sanity Suite (Top 30% P1+P2), and Full Regression (100%).
Pass 3 — Traceability Linkage	Binds tests to parent requirement IDs (`REQ-xxx`) and checks for orphaned clauses with zero coverage.

Polyglot Automation Script Generation (Page Object Model)

Plain-English test cases are synthesized into production-ready test automation scripts across Playwright (TypeScript), Cypress (JavaScript), Selenium WebDriver (Java/Python), and PyTest. Scripts strictly implement the Page Object Model (POM), resilient role-based selectors (`getByRole`, `data-testid`), and explicit asynchronous assertions (`toBeVisible()`) to eliminate test flakiness.

11. Reviewing Execution & Telemetry Results

- **Inspect Telemetry Counters**: Verify the real-time breakdown of Passed, Failed, Blocked, and Not Executed test cases.
- **Review Priority Distribution**: Confirm that all critical P1 and P2 tests pass before reviewing non-critical P3 tests.
- **Audit Requirement Coverage**: Verify that the Requirement Traceability Matrix demonstrates 100% functional requirement coverage.
- **Examine Failure Diagnostics**: Click on failing test cards in the runner to inspect step logs, actual vs. expected results, and simulated stack traces.

12. Complete System Demonstration Procedure

Use the following sequence when presenting the project to an examiner or during a technical team evaluation.

ORDER	DEMONSTRATION ACTIVITY
1	Introduce the project and explain its role in shift-left testing, automated requirement analysis, and CI/CD governance.
2	Start the backend API server (`npm run dev` in `server/`) or verify the live Render cloud service.
3	Open the Testing Dashboard at `http://localhost:5173` or production URL `https://generative-ai-based-intelligent-aut-delta.vercel.app/`.
4	Click **"Sign In with Demo Account"** to bypass credential entry and demonstrate project workspace isolation.
5	Navigate to **"SRS Input"**, paste a software requirement or upload a document, and click **"Generate Test Cases"**.
6	Open **"Ambiguity Detection"**, show flagged untestable phrases, and click **"Replace in Requirement"** to apply testable metrics.
7	Demonstrate **"Negative & Edge Cases"**, synthesize mock data in **"AI Test Data Generator"**, and run **"Duplicate Detection"**.
8	Open **"Requirement Traceability Matrix"** to prove 100% coverage, and use **"Requirement Change Impact"** to show regression diffing.
9	Generate a Playwright automation script, trigger **"Live Test Execution"** to observe real-time telemetry, and show the 12 CI/CD REST endpoints.

13. Shutting Down the System

- Close the browser tabs used for the Testing Dashboard and API documentation.
- In the frontend terminal, press `Ctrl + C` to stop the Vite development server.
- In the backend terminal, press `Ctrl + C` to stop the Express server.
- Close all terminal windows after the servers have cleanly exited.

14. Troubleshooting

PROBLEM	WHAT TO CHECK
Backend API does not start	Verify port 5001 is not in use by another process. Run `npm run dev` from the `server/` folder and inspect terminal logs.
MongoDB connection failure	Confirm that `MONGO_URI` exists in `server/.env` and that your current IP address is whitelisted in MongoDB Atlas.
External AI API quota exceeded	The system automatically falls back to its built-in heuristic NLP rule engine to synthesize tests locally with zero downtime.
Frontend fails to load	Confirm Node.js is installed, run `npm install` in the root directory, and ensure `npm run dev` is running.
Browser page shows stale cache	Perform a hard browser reload (`Ctrl + F5` or `Cmd + Shift + R`) to clear client-side cache.

15. Operational Notes and Quick Checklist

Operational Notes

- Keep API keys and database credentials in local `.env` configuration files. Do not commit or publish credentials to public repositories.
- Always run backend commands from the `server/` directory and frontend commands from the project root directory.
- Keep both terminal windows open simultaneously when running the system locally in development mode.
- Review the Requirement Traceability Matrix before releasing new software builds to guarantee zero untested requirement clauses.

Quick Operating Checklist

CHECK	READY
Node.js and npm installed and available in terminal	■
Backend dependencies installed in `server/` folder	■
Gemini API key and MongoDB Atlas URI configured in `server/.env`	■
Express backend running on port 5001 or connected to Render API	■
Frontend dependencies installed in root application directory	■
Vite development server running on port 5173 or live Vercel URL accessible	■
Demo account login verified and project workspace selected	■
SRS ingested, ambiguities resolved, and test cases generated	■
Automation scripts generated and live execution telemetry reviewed	■

16. Key Terms for Viva

TERM	MEANING
Shift-Left Testing	Moving quality assurance activities early to the requirement phase to catch defects before code is written.
IEEE 829	International software standard defining the structure of test cases, preconditions, steps, and expected outcomes.
Boundary Value Analysis	A black-box testing technique targeting boundary extremes (Min, Max, Min-1, Max+1) where defects concentrate.
Levenshtein & Jaccard	Mathematical string and token distance metrics used to eliminate duplicate test cases from test suites.
Page Object Model (POM)	Design pattern in test automation that separates web page elements from test logic for maintainable scripts.
RTM Matrix	Bidirectional grid proving 100% compliance between business requirements, test cases, and execution verdicts.
Heuristic Fallback Engine	Internal rule-based NLP parser ensuring zero downtime when external generative AI APIs are unreachable.
JWT & bcrypt	Stateless token authentication and one-way cryptographic password hashing for enterprise access control.
Playwright / Selenium	Industry-leading polyglot browser automation frameworks for end-to-end regression testing.
CI/CD Webhooks	Automated HTTP REST triggers that launch test generation and regression execution on every `git push`.

END OF OPERATING PROCEDURE